

현장의 신호가 운영의 판단이 되기까지.

PLC 생산 신호 생성부터 Gateway 표준화, OEEAnalyzer의 품질 기반 기간 계산, TwinForge의 MES 맥락과 운영 화면, DataNexus의 독립 관측까지 다섯 제품의 실행 흐름과 장애 처리 기준을 설명합니다.

05

CONNECTED PRODUCTS

2

INDEPENDENT DATA PLANES

A×P×Q

QUALITY-AWARE RESULT

다섯 제품, 제어와 분석을 분리한 데이터 흐름

계측 경로는 PLC → Gateway → OEEAnalyzer로 흐르고, 업무 경로는 TwinForge → OEEAnalyzer로 들어갑니다. 계산 결과는 TwinForge 운영 화면으로 돌아가고, DataNexus는 Kafka 표본과 OEE Window를 별도 권한으로 읽어 전체 흐름을 관측합니다.

01 · ACQUIRE

PLC Simulation

프로토콜 프레임과 레지스터 값을 생성하고 태그 읽기·쓰기를 재현합니다.

Modbus · MELSEC · FINS · S7 · MQTT

02 · NORMALIZE

IIoT Gateway

장치 주소를 표준 경로로 만들고 품질, 알람, 이력과 OPC UA를 제공합니다.

Cache · Alarm · Historian · HA

03 · ANALYZE

OEEAnalyzer

네 생산 신호와 작업 맥락을 결합해 기간별 A/P/Q/OEE와 계산 신뢰도를 만듭니다.

Kafka · TimescaleDB · Coverage

04 · OPERATE

TwinForge

작업지시 맥락을 보내고 External 분석 결과를 MES·설비·3D 운영 화면과 함께 표시합니다.

SignalR · MES · Workflow · Three.js

05 · OBSERVE

DataNexus

Kafka 표본과 권위 있는 OEE Window를 독립적으로 읽어 연결·품질·계보를 교차 확인합니다.

Read-only · Manual commit · DLQ

생산 신호 생성

PLC Simulation이 Total Count 40020, Reject Count 40022, Machine State 40024와 Counter Epoch 40025를 Modbus 메모리에서 갱신합니다.

수집과 표준화	Gateway가 네 주소를 읽고 <code>DataAccess.DockerModbus.PlcSimulation.*</code> OPC UA 노드로 제공하며 값·StatusCode·Source Timestamp를 유지합니다.
이벤트 보존	OEE Worker가 네 노드를 한 번에 읽어 하나의 sample로 정규화하고 <code>oee.integration.v1</code> Kafka 토픽으로 전달합니다.
작업 맥락 동기화	TwinForge가 활성 작업지시, 제품, UTC 구간과 ideal cycle을 같은 <code>equipmentKey</code> 의 production context로 전송합니다.
기간 OEE 계산	OEEAnalyzer가 계획정지를 제외한 구간에서 가동시간과 카운터 증가분을 계산하고 coverage·status·reasons·revision을 함께 저장합니다.
운영 화면 반영	TwinForge가 최신 window를 다시 조회해 External A/P/Q/OEE, coverage, 작업지시와 context ID를 MES 화면에 표시합니다.
독립 관측과 대조	DataNexus가 전용 consumer group으로 같은 Kafka 표본을 읽고 OEEAnalyzer REST의 최신 Window를 조회해 품질, Kafka 위치, coverage와 revision을 함께 제시합니다.

REPRODUCIBLE DOCKER SCENARIO

TwinForge command → PLC Ack/feedback + production evidence → OEE window → TwinForge + DataNexus

통합 스크립트는 Start·Stop·Reset sequence와 Ack, SpeedSetpoint 1.8 m/s와 PLC 소유 SpeedPV, 네 Gateway 생산 경로의 Good 품질, OEEAnalyzer context·coverage, DataNexus의 Kafka·OEE API 연결과 rejected 0을 순서대로 검사합니다.

PLC Simulation

제조사별 통신 구현을 DLL 플러그인으로 분리하고 DriverHost가 설정, 연결, 태그와 최신 값을 하나의 실행 모델로 관리합니다.

The screenshot displays the PLC Simulator interface. On the left is a sidebar with navigation options: Dashboard, PLC Nodes, Driver Configuration, Tag Monitor (selected), Data Mapping, Logs & Events, and Settings. The main area is titled 'LIVE TAG TREE, VALUES, AND EDITOR' and 'Simulation and Tag Monitor'. It is divided into two panels: 'Tag Explorer' on the left and 'Data Monitor & Editor' on the right. The 'Tag Explorer' shows a tree structure for a 'Modbus TCP Device Simulator' with various tags like Coil, Valve_CTRL, Control, Ack_Reason, Ack_Sequence, Ack_Status, Command_Sequence, Command_Type, Discrete_Input, Run_FB, and Holding_Register. The 'Data Monitor & Editor' panel shows a table of current values and timestamps for these tags. A 'Refresh' button is located in the top right of this panel.

ADDRESS	NAME	DRIVER	TYPE	VALUE	QUALITY	TIMESTAMP	
00001	Valve_CTRL Coil	Modbus TCP Device Simulator	Bool	true	GOOD	0시 43분 58초.398	Write
40016	Ack_Reason Control	Modbus TCP Device Simulator	Int16	0	GOOD	0시 43분 58초.398	Read only
40013	Ack_Sequence Control	Modbus TCP Device Simulator	Int16	2	GOOD	0시 43분 58초.398	Read only
40014	Ack_Status Control	Modbus TCP Device Simulator	Int16	3	GOOD	0시 43분 58초.398	Read only
40012	Command_Sequence Control	Modbus TCP Device Simulator	Int16	2	GOOD	0시 43분 58초.398	Write
40011	Command_Type Control	Modbus TCP Device Simulator	Int16	0	GOOD	0시 43분 58초.398	Write
40015	Control_Mode Control	Modbus TCP Device Simulator	Int16	1	GOOD	0시 43분 58초.398	Read only

LIVE TAG MONITOR 읽기·쓰기·품질 상태를 WebSocket으로 실시간 표시

STARTUP

플러그인 검색과 프로파일 적용

시작 시 공통 `IPlcDriver` 구현을 DLL에서 찾습니다. PLC 모델을 선택하면 프로토콜, 모델, 통신 모듈 기본값을 병합하고 드라이버를 재구성한 뒤 태그를 검색합니다.

READ LOOP

1초 주기 폴링

`MonitorBackgroundService`가 활성 드라이버를 순회합니다. 연결이 끊긴 드라이버는 재연결하고, 발견한 태그 목록을 읽어 ID별 최신 값과 품질을 갱신합니다.

WRITE PATH

태그 쓰기와 확인

API가 태그 ID 또는 주소를 찾고 드라이버가 데이터 형식과 PLC 바이트 순서에 맞춰 값을 변환합니다. 성공 응답 뒤 캐시를 GOOD 품질로 갱신합니다.

LIVE UI

WebSocket 스냅샷

`/ws/monitor`가 드라이버 상태, 태그 값, 데이터 매핑과 통신 로그를 전송합니다. 화면은 전체 새로고침 없이 연결 배지와 숫자를 갱신합니다.

읽기 순서

1. 활성 드라이버의 연결 상태를 확인하고 필요하면 재연결합니다.
2. 태그가 없으면 드라이버에서 주소와 데이터 형식을 검색합니다.
3. 프로토콜별 Read 프레임을 보내고 응답을 공통 TagValue로 변환합니다.
4. 태그 ID별 최신 값, 품질, 시각을 저장하고 WebSocket으로 전달합니다.

오류 처리

한 드라이버의 통신 실패는 로그에 남기고 다른 드라이버 폴링은 계속합니다. 비활성 드라이버 쓰기와 존재하지 않는 태그 요청은 서버에서 거부하며, 다음 폴링 주기에 재연결을 시도합니다.

OEE 생산 신호

Docker 데모의 Modbus 드라이버는 기존 속도·온도 태그와 분리된 누적 생산 계약을 제공합니다. Total Count와 Reject Count, Counter Epoch는 2-register Int32이고 Machine State는 Int16입니다. Counter Epoch가 바뀌면 Analyzer는 이전·이후 증가분을 이어 붙이지 않습니다.

02 · EDGE INTEGRATION

IIoT Gateway

Channel·Device·Tag 설정을 실제 통신으로 변환하고, 수집된 하나의 실시간 캐시를 OPC UA·알람·Historian이 공유합니다.

IIoT Gateway

Data Access

Historian

Aggregation

Alarm & Condition

Monitoring

System

Data Access Configuration

Guide Engine Running

Data Access / Channel List / DockerModbus / PlcSimulation

PlcSimulation / Tags

Communication tags refresh values from the device. Calculated values belong in Custom Tags.

← Back

+ Add Tag

TAG NAME	SOURCE	ADDRESS	DATA TYPE	SCAN RATE	ACCESS	ACTIONS
Ack_Reason	Communication	40016	Int16	500 ms	R	
Ack_Sequence	Communication	40013	Int16	500 ms	R	
Ack_Status	Communication	40014	Int16	500 ms	R	
Command_Sequence	Communication	40012	Int16	500 ms	R/W	
Command_Type	Communication	40011	Int16	500 ms	R/W	
Control_Mode	Communication	40015	Int16	500 ms	R	
Counter_Epoch	Communication	40025	Int32	500 ms	R	

HIERARCHICAL TAG MODEL

주소·형식·권한·Scan Rate를 장치 구조로 관리

CONFIGURATION Channel → Device → Tag Channel이 드라이버를 선택하고 Device가 엔드포인트를, Tag가 PLC 주소, 형식, 권한과 Scan Rate를 정의합니다. Repository가 이 구성을 MySQL에 저장합니다.	ACQUISITION Scan Rate 기반 수집 Active 노드가 읽을 시점이 된 태그만 선택합니다. 드라이버 응답의 값, 품질, 형식과 Source Timestamp를 표준 DataAccess 경로에 저장합니다.
ALARM + HISTORY 캐시에서 파생되는 기능 알람은 1초마다 HiHi·High·LoLo·Low를 평가하고 상태 전환 때만 이벤트를 냅니다. Historian은 최소 100ms 주기로 값을 샘플링합니다.	OPC UA + HA 안전한 외부 제공 OPC UA가 계층형 주소 공간과 인증서 정책을 제공합니다. Active/Standby와 fencing token은 수집, 서버와 저장이 한 노드에서만 실행되도록 제어합니다.

수집에서 OPC UA까지	1. 설정 저장소에서 활성 Channel, Device와 통신 태그를 읽습니다. 2. Driver Key로 프로토콜 DLL을 선택하고 장치 연결 옵션을 구성합니다. 3. 성공한 값을 <code>DataAccess.Channel.Device.Tag</code> 경로의 캐시에 기록합니다. 4. OPC UA Read가 같은 캐시를 조회해 값과 품질을 외부 클라이언트에 제공합니다.
품질과 장애 처리	PLC 읽기 실패 시 해당 태그를 Bad 품질로 바꾸어 오래된 값을 정상으로 오해하지 않게 합니다. Historian cache miss도 0으로 저장하지 않습니다. 설정 변경 신호가 발생하면 OPC UA 서버가 주소 공간을 다시 생성합니다.
OEERAnalyzer 제공 경로	통합 binding은 namespace URI http://examples.org/UA/DataAccess/를 세션의 namespace table에서 해석하고 Total 40020, Reject 40022, State 40024, Epoch 40025 노드를 함께 읽습니다. 이때 값만 넘기지 않고 각 노드의 StatusCode와 Source Timestamp를 계산 입력에 포함합니다.

OEEAnalyzer

Gateway의 네 생산 신호와 TwinForge의 MES 맥락을 이벤트 시간 기준으로 결합합니다. 계산값과 함께 coverage, 상태, 원인과 revision을 반환해 결과를 사용할 수 있는 조건까지 설명합니다.

e-data recalculation visible.

STATUS	A / P / Q	OEE	COVERAGE	PRODUCTION	RE
Final	100.0% / 88.0% / 100.0%	88.0%	100.0%	2 total 0 reject	r2 26. 12

AUTHORITATIVE OEE CONTROL ROOM

설비·UTC 구간을 조회하고 계산값과 신뢰도를 함께 확인

ACQUISITION 네 노드 batch read Worker가 1초 목표 주기로 Total, Reject, State와 Epoch를 같은 OPC UA 세션에서 읽습니다. namespace index를 고정하지 않고 namespace URI를 연결마다 해석합니다.	EVENT CONTRACT 품질과 시각을 포함한 sample equipmentKey, source instance·mode, mapping version, 원본·수신 UTC, 네 StatusCode와 source path를 eventId와 함께 Kafka에 보존합니다. UInt64 범위 카운터는 문자열로 직렬화합니다.

EVENT-TIME WINDOW

계획 구간과 카운터 증가분

계획정지를 제외한 시간에서 Running 초를 합산하고 동일 epoch·source·mapping의 Good 표본 사이 증가분만 생산·불량으로 인정합니다.

EXPLAINABLE RESULT

숫자와 신뢰도 동시 반환

$A = \text{Run} / \text{Planned}$, $P = \Delta\text{Total} \times \text{IdealCycle} / \text{Run}$, $Q = (\Delta\text{Total} - \Delta\text{Reject}) / \Delta\text{Total}$ 을 계산하고 OEE와 coverage, status, reasons, revision을 같은 window로 저장합니다.

Production context 계약

1. TwinForge가 MES 시작 이후 수집된 표본을 조회해 Source Timestamp를 계산 시작 경계로 선택합니다.
2. `twinlineforge-{orderId}` context에 equipment key, order, product, UTC 시작·종료, ideal cycle, planned stops, Source Mode와 version을 담습니다.
3. `PUT /api/integration/v1/production-contexts/{contextId}`에 `Idempotency-Key: contextId:version`을 보냅니다.
4. 같은 version의 내용 충돌은 409, 기존 context와 기간이 겹치면 422로 거부됩니다.

완전성과 상태 판정

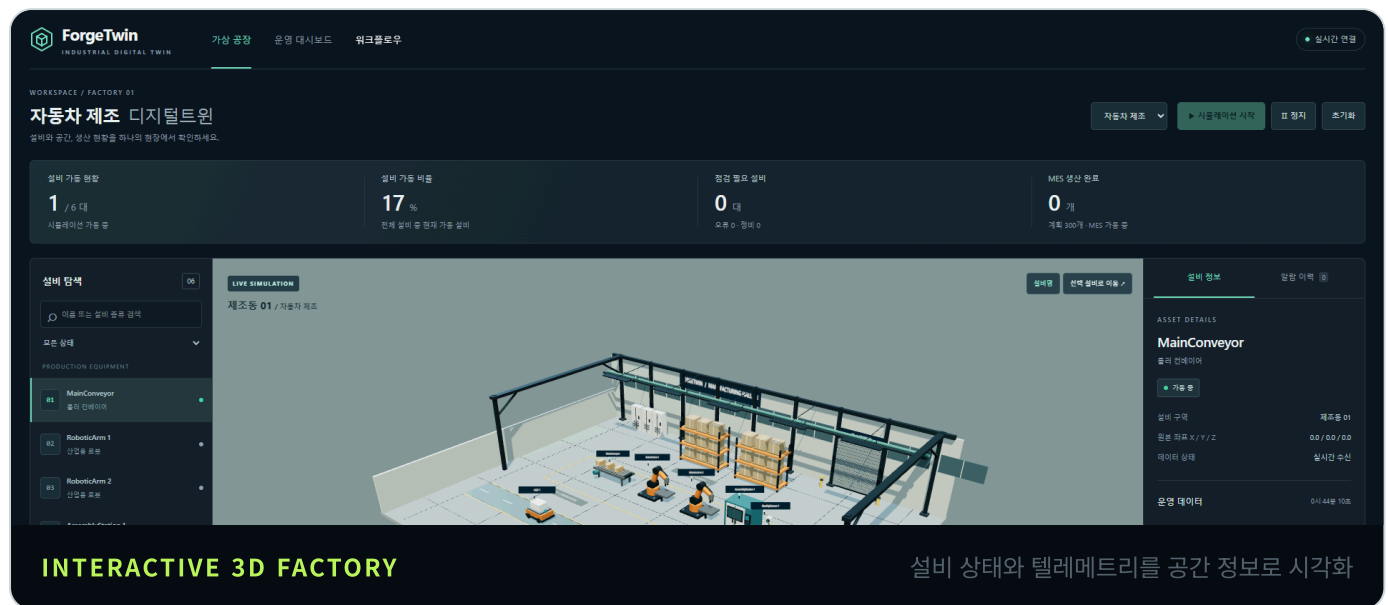
Good 표본이 stale 허용시간을 넘거나 timestamp가 충돌하고 epoch·source·mapping이 바뀌면 해당 구간 또는 증가분을 제외합니다. 부분 coverage에는 정상 OEE 값을 만들지 않습니다. 비율 위반은 Invalid, 데이터 근거가 부족하면 Incomplete, 늦은 이벤트 허용시간 전은 Provisional, 계산 분모가 없으면 NotApplicable, 조건을 모두 통과한 닫힌 구간은 Final입니다.

API 권한과 조회

모든 integration API는 JWT 인증 뒤 `equipment_key` claim과 기능별 scope를 검사합니다. 조회에는 `integration:read`, 맥락 저장에는 `integration:context`, 직접 sample 입력에는 `integration:ingest`가 필요합니다. 슬래시가 포함된 설비 키는 reverse proxy에서 깨지지 않도록 query parameter 형태도 제공합니다.

TwinForge

100ms 장비 시뮬레이션, OPC UA 외부 신호, MES와 실행 그래프를 하나의 장비 모델과 이벤트 경로에 연결하고 OEEAnalyzer의 External 결과를 작업 맥락과 함께 표시합니다.



SIMULATION

100ms 장비 틱

Industry Factory가 장비와 위치를 만들고 엔진이 각 장비의 Update를 호출합니다. 값, 위치, 상태와 알람 변경은 Event Bus에 발행됩니다.

REAL TIME

SignalR과 3D 동기화

이벤트를 WebSocket으로 받은 Zustand Store가 설비 카드와 3D Scene의 공통 상태를 갱신합니다. 최근 텔레메트리 20개와 알람 100개를 유지합니다.

MES

작업지시와 생산 진행

우선순위에 따라 최대 2개 작업을 동시에 Release합니다. 공정 진행에 따라 양품, 불량, WIP, OEE와 스테이션 상태가 갱신됩니다.

EXTERNAL OEE

맥락 동기화와 분석 결과

Released 또는 InProgress 작업 중 우선순위가 높은 작업을 선택해 OEEAnalyzer에 맥락을 보냅니다. 반환된 A/P/Q/OEE, coverage와 reasons는 자체 MES OEE와 구분된 패널에 표시합니다.

WORKFLOW

위상 정렬 실행

노드를 연결 순서로 정렬하고 선행 Output을 후속 Input으로 전달합니다. 각 Executor의 Running, Success, Error 상태와 로그를 표시합니다.

외부 OPC UA 적용

1. OPC UA Client가 200ms Publishing Interval로 Gateway 노드를 구독합니다.
2. Equipment Binding이 노드 경로와 Twin 장비 속성을 연결합니다.
3. 정상 숫자에 $\text{value} \times \text{scale} + \text{offset}$ 을 적용합니다.
4. 엔진이 값을 적용하고 SignalR 이벤트로 카드와 3D 오브젝트를 갱신합니다.

데이터 품질

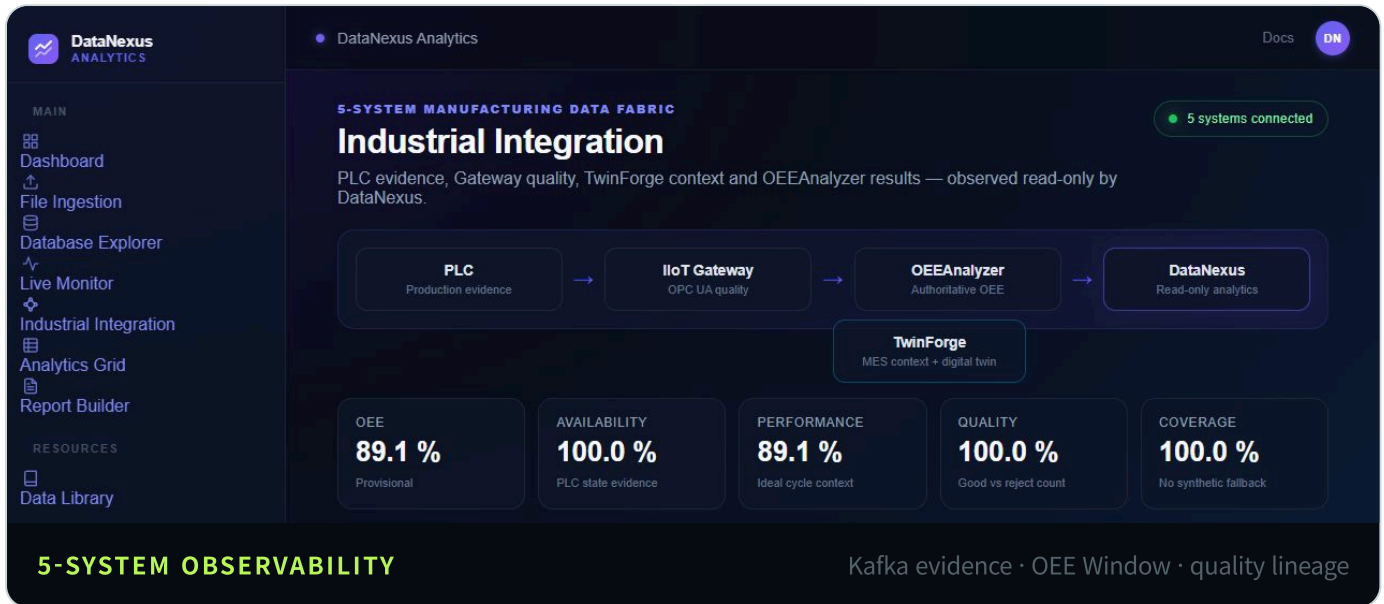
연결 실패, 초기값 대기과 형식 오류를 서로 다른 Bad 품질로 구분합니다. 외부 모드에서는 내부 시뮬레이션 Update가 수신 값을 덮어쓰지 않습니다.

OEEAnalyzer 동기화

Docker 프로필의 백그라운드 서비스는 3초마다 활성 MES 작업과 latest sample 경계를 맞추고 context를 upsert한 뒤 현재 UTC까지의 OEE window를 조회합니다. 사용자는 **Sync now**로 같은 과정을 즉시 실행할 수 있고, 브라우저는 통합 상태를 5초마다 갱신합니다. 결과가 Incomplete 또는 AwaitingSamples이면 충분한 Good 표본이 모일 때까지 OEE를 확정값처럼 표시하지 않습니다.

DataNexus Analytics

제어 경로에 참여하거나 OEE를 다시 계산하지 않는 읽기 전용 분석 소비자입니다. Kafka의 원본 생산 표본과 OEEAnalyzer REST의 권위 있는 Window를 함께 보여주어 연결 상태, 데이터 품질, 계산 결과와 계보를 한 화면에서 대조합니다.



KAFKA EVIDENCE

독립 consumer와 수동 commit

`datanexus-ooo-read-v1` 전용 group으로 `ooo.integration.v1`을 소비합니다. OEE 처리 Worker와 partition을 나누지 않으며, `key·payload·schema·설비 허용 목록`을 검증한 뒤에만 offset을 commit합니다.

AUTHORITATIVE RESULT

OEEAnalyzer Window 읽기

JWT 로그인과 토큰 갱신으로 읽기 전용 API를 호출합니다. A/P/Q/OEE를 다시 계산하지 않고 `coverage`, `status`, `reasons`, `revision`, `context`와 함께 원본 결과를 보존합니다.

QUALITY + LINEAGE

값보다 근거를 먼저 표시

source·received UTC, OPC UA 품질, schema·mapping version, Kafka partition·offset을 표본과 함께 유지합니다. Window가 Incomplete이거나 OEE가 null이면 정상값처럼 채우지 않고 N/A와 원인을 표시합니다.

FAILURE ISOLATION

DLQ와 의존성 상태

잘못된 payload는 위치와 사유를 포함해 idempotent DLQ에 보존합니다. Kafka와 OEE API 연결 상태를 분리해 한쪽 장애가 다른 경로의 정상처럼 보이지 않으며, DataNexus 장애는 생산 제어와 OEE 계산을 멈추지 않습니다.

수집 순서

1. Kafka key와 payload의 equipment key, v1 schema, UTC 시각, 품질과 필수 경로를 검사합니다.
2. 정상 표본의 설비 상태·카운터·source age·Kafka position을 저장하고 SignalR로 화면에 전달합니다.
3. OEEAnalyzer에서 같은 설비의 최신 Window를 조회해 context와 revision을 포함한 분석 결과를 갱신합니다.
4. 화면이 실시간 표본과 기간 Window의 출처·시각을 구분해 표시합니다.

권한 경계

DataNexus에는 **integration:read**와 허용된 **equipment_key**만 부여합니다. PLC·Gateway 쓰기, OEE sample/context 쓰기와 TwinForge 제어 API는 역할 범위 밖이며, 분석 화면이 생산 제어의 새로운 우회 경로가 되지 않도록 합니다.

검증 근거

Release 빌드 경고 0·오류 0, DataNexus 단위 테스트 14/14, Docker Compose 구성 검증과 5자 통합 runner를 통과했습니다. 수용 기준은 Kafka connected, OEE API connected, latest sample/window 존재, coverage 1.0, rejected 0입니다.

통합 Docker 실행 주소와 검증 범위

PLC Simulation	http://localhost:5088	Web UI · Modbus TCP 1502
IIoT Gateway	http://localhost:18080	UI/API · OPC UA 17312
OEERAnalyzer	http://localhost:18083	React UI · Web API 18082
TwinForge	http://localhost:18081	MES · External OEE · Workflow · 3D factory
DataNexus	http://localhost:18084	Kafka evidence · OEE Window · health

OEE 통합 binding은 Docker 환경 변수로 적용되며 Processing Rules 화면의 3-tag OEE Logic Builder와 역할이 다릅니다. Gateway 설정 화면 일부는 인메모리 구성 모드에서 촬영했고, TwinForge 워크플로 Run은 브라우저 Store 기반 실행 시뮬레이션입니다. External OEE와 DataNexus 패널은 OEERAnalyzer API의 실제 context·window 결과를 사용하며 DataNexus는 제어 또는 OEE 재계산에 참여하지 않습니다.